Abstract Data Type (ADT)

ADT

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

p->next_ = (node_t*)malloc(sizeof(node_t)); p->next_->key = 20; p->next_->next_ = NULL;

p = p->next; // comment
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) Results in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

p->next_ = (node_t*)malloc(sizeof(node_t)); p->next_->key = 20; p->next_->next_ = NULL;

p = p->next; // comment
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) Results in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

P->next_ = (node_t*)malloc(sizeof(node_t)); p->next_->key_ = 20; p->next_->next_ = NULL;
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) Results in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

P->next_ = (node_t*)malloc(sizeof(node_t)); p->next_->key_ = 20; p->next_->next_ = NULL;
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) Results in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10; free(p); // p ?
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) p in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10; free(p); // p ?
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) p in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

p->next_ = (node_t*)malloc(sizeof(node_t)); p->next_->key_ = 20;
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) P->next_->next_ in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

p->next_ = (node_t*)malloc(sizeof(node_t)); p->next_->key_ = 20;
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) P->next_->next_ in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

p->next_ = p;
```

a) Runtime error

b) p->next_->key_ is 10

c) P->next_->next_ in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

p->next_ = p;
```

a) Runtime error

b) p->next_->key_ is 10

c) P->next_->next_ in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Concept of Type

- An integer in Mathematics is a set
- t is closed for addition and multiplication
- 25 + 34 is 59
- How do we add?
    - Add unit digits and then digits in the tenth place and so on along with the carry from the earlier stage
    - Add row wise
    - Use fingers and toes
    - Use tally marks
    - Convert to some other base, add and convert the result back to decimal
- Will the result be different?
- Should we learn how to add if the integers have different values?

© Confidential Information of PES University. All Rights Reserved - 2026

Concept of Type

- An integer is a type
- Represented in a programming language normally as a subset
- A type indicates
    - What values are legal
    - What we can do with them
    - Does not say how we can do them
    - Allows us treat almost all values of that type uniformly
- Type of a variable in 'C' has to be declared in the beginning
- Once declared, cannot be changed.
- Type of a variable is determined at compile time
- In Python,
    - Type is associated with a value.
    - A variable gets a type based on type of value assigned.
    - Type is dynamic.

© Confidential Information of PES University. All Rights Reserved - 2026

Type and a programming language

- Can a language support all possible types of the real world?
- A car is a type.
- Supports the following operations
    - Start
    - Stop
    - Accelerate
    - Brake
    - Steer
- Once you have learnt driving a car, you can (almost) drive any other car
- Other types of real world : fan, mobile phone, browser, laptop, novel, dream, ...
- Can a language support all possible types of the real world?
- NO

© Confidential Information of PES University. All Rights Reserved - 2026

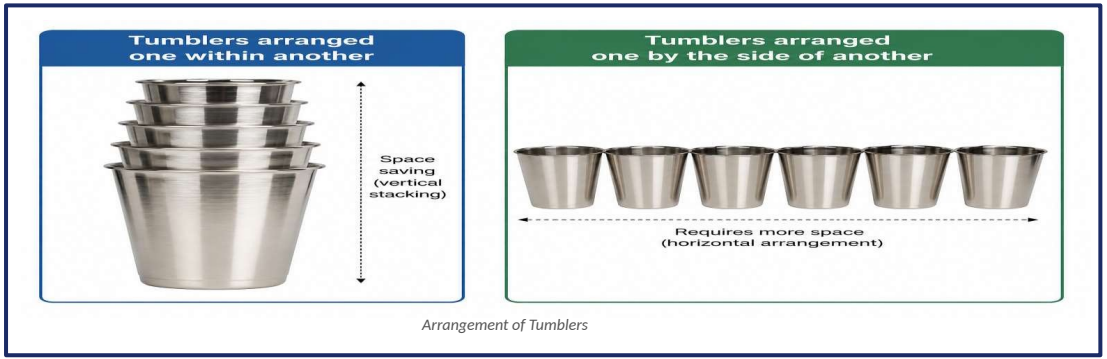
Type and a programming language

- A Language should support a way of creating our own type
- Abstract Data Type (ADT)
    - Like a type (int, double, …) in a language, this supports
        - A few operations
        - Specifies what they do and not how they do
        - Specifies the interface and not the implementation
        - Both the user and the implementor depend on the interface
        - Interface once decided is sacred; cannot be changed
        - Implementation can be changed without affecting the user
        - This is called inversion of dependency
- If B depends on A, any change to A would require change to B.
- If both A and B depend on what does not change, change to A will not affect B

© Confidential Information of PES University. All Rights Reserved - 2026

Data Structure

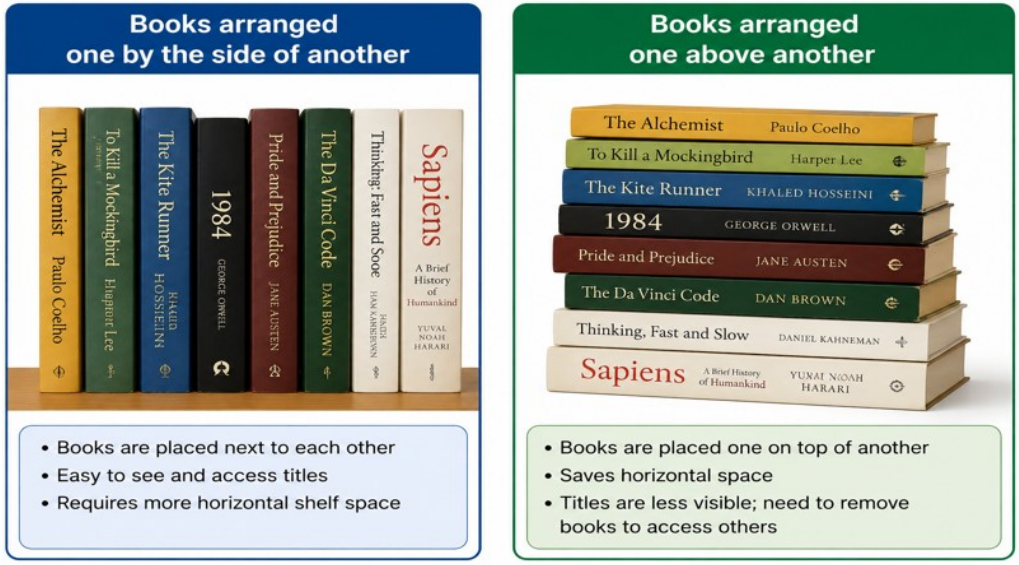
- A data structure is an abstract data type
- Has number of components arranged in some suitable manner
    - To achieve some goal
- An example: arrangement of Tumblers



Arrangement of Tumblers

© Confidential Information of PES University. All Rights Reserved - 2026

Data Structure



Another Example: Arrangement of Books

© Confidential Information of PES University. All Rights Reserved - 2026

List

- Is a data structure
- Has zero or more components
- Each element has an ordinal position
- Cannot directly access the element based on position
    - Sequential access
    - No random or direct access
- Insertion and deletion once the position is known requires
    - A few operations

© Confidential Information of PES University. All Rights Reserved - 2026

List

- Interface :
  - Create
  - Destroy
  - Insert
  - Delete
  - …
- Implement using
  - A linked list
  - An Array

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)